

3 | Codierung

Wenn wir ein Foto anschauen, sehen wir ein Bild. Wenn wir ein Lied hören, hören wir Musik. Ein Computer nimmt diese Dinge jedoch nicht so wahr wie wir. Er hat ja keine Augen oder Ohren. Für ihn besteht ein Foto, ein Lied oder ein Video aus einer langen Folge von 0 und 1, weil das ist ja seine Sprache. Jede einzelne 0 oder 1 ist dabei ein **Bit**.

Damit wir aber nicht solch für unsere Augen nichts aussagenden Ansammlungen von 0 und 1 anschauen müssen, übersetzt der Computer diese binären Anordnungen in Pixel oder Buchstaben. Diese Übersetzung zwischen menschenverständlichem und computerverständlichem bezeichnet man als **Codierung** und **Decodierung**.



1. Textcodierung

Bei der Textcodierung geht es also darum, wie der Computer Text (Buchstaben, Emojis, ...) speichern, darstellen und/oder übertragen möchte. Bevor wir unsere heutigen Computer hatten, gab es schon erste Technologien, die fähig waren, Buchstaben und Zahlen über Ferne zu übertragen. Schauen wir uns mal folgendes Beispiel an:

Das Fernschreibgerät mit Telex



Das Gerät das Sie links sehen ist ein Fernschreibgerät, das mit dem Telex System mittels elektrischer Signale weltweit Nachrichten austauschen konnte. Quasi ein Vorgänger der schriftlichen Kommunikation, wie wir sie heute auf unseren Smartphones kennen. Unten an der Tastatur konnte der Benutzer eine Nachricht schreiben, welche dann an einen anderen Benutzer gesendet werden konnte. Der konnte dann die Nachricht ausgedruckt auf seinem Stück Papier sehen. Aber was passierte zwischen Schreiben und Drucken im Hintergrund?

Um diese Abfolge zwischen geschriebenem Buchstaben, Übertragung und ausgedrucktem Buchstaben zu verstehen, betrachten Sie die Abbildung 1.

Die Tabelle zeigt, wie die einzelnen Zeichen codiert wurden. Schwarz bedeutet, dass Strom fließt, Weiss bedeutet, dass kein Strom fließt. Wenn Sie beispielsweise den Buchstaben A eingeben, sendet der Fernschreiber zunächst ein Startimpuls. Dieses signalisiert dem empfangenden Gerät, dass nun ein neues Zeichen übertragen wird. Anschliessend werden fünf Bits übertragen. Beim A lautet die Folge:

Strom – Strom – kein Strom – kein Strom – kein Strom

Zum Abschluss folgt ein Stoppimpuls, das das Ende des Zeichens signalisiert.

Der empfangende Fernschreiber liest diese Folge und vergleicht sie mit derselben Codetabelle. Dadurch erkennt er, dass die übertragene Zeichenfolge für den Buchstaben A steht, und kann diesen ausdrucken. Danach beginnt die Übertragung des nächsten Zeichens.

Dieses Verfahren wird als Start-Stopp-Verfahren bezeichnet. Die Buchstaben wurden also nicht direkt übertragen, sondern zunächst in eine Folge von zwei möglichen Zuständen (Strom oder kein Strom) **codiert**.

Aufgabe 1

Für einen Buchstaben gibt es 1 Startimpuls, 5 Impulse (5 Bit) für das Zeichen und 1.5 Stoppimpulse (eigentlich kein Bit, sondern eine Zeit). Das Ganze dauert 150ms. Wie lange dauert ein einzelner Impuls? Wie viele Zeichen pro Minute sind möglich?

NR. VAN DE COMBI- NATIE	LETTERS	CIJFERS	NR. VAN DE ELEMENTEN						
			START	1	2	3	4	5	STOP
1	A	—							
2	B	?							
3	C	:							
4	D								
5	E	3							
6	F								
7	G								
8	H								
9	I	8							
10	J	DEL							
11	K	(							
12	L	)							
13	M	.							
14	N	*							
15	O	9							
16	P	0							
17	Q	!							
18	R	4							
19	S	'							
20	T	5							
21	U	7							
22	V	=							
23	W	2							
24	X	/							
25	Y	6							
26	Z	+							
27	TERUGLOOP WAGEN								
28	NIEUWE REGEL								
29	LETTERS								
30	CIJFERS								
31	TUSSENRUIMTE								
32	5 x STROOMLOOS								

Abbildung 1: Der Telex Code

Der Morsecode

Ein anderes System, um Texte über weite Strecken zu senden ist der Morsecode. Jeder Buchstabe wird durch eine Folge aus Punkten und Strichen dargestellt. Ein Punkt steht für ein kurzes Signal, ein Strich für ein langes Signal (siehe die Beispielbuchstaben in Abbildung 2). Die Zeichen sind unterschiedlich lang, ein E besteht beispielsweise aus einem einzelnen Punkt und das J aus einem Punkt und drei Strichen.

A — • —	G — • •	U • • —
B — • • •	H • • • •	V • • • —
C — • — •	I • •	W • — —
D — • •	J • — —	X • — • —
E •	K • — •	Y • — —
F • • — •	L • — • •	Z — — • •

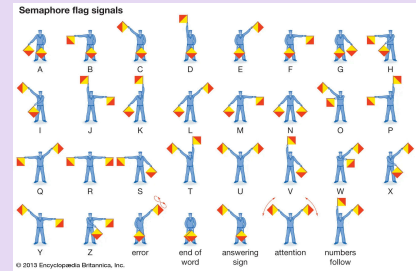
Abbildung 2: Morsecode

Andere historische Textcodierungen

Das Telex System oder der Morsecode waren nicht die einzigen, die es Menschen ermöglichte, Texte über weite Strecken zu senden. Der **Semaphor Code** wurde zum Beispiel in der Schifffahrt verwendet, um zwischen Booten über weitere Distanzen zu kommunizieren. Die Buchstaben werden durch unterschiedliche Positionen von zwei Flaggen dargestellt. Jede Stellung der Arme entspricht einem bestimmten Buchstaben (siehe rechts).



Ein ähnliches Prinzip wurde auch bei den optischen Telegrafen verwendet. In Frankreich entstand Ende des 18. Jahrhunderts ein Netz aus Telegrafentürmen, die über grosse Distanzen miteinander verbunden waren. Auf jedem Turm wurden bewegliche Signalarme in verschiedene Positionen gebracht. Die nächste Station beobachtete das Signal und gab es an die nächste Station weiter. Auf diese Weise konnten Nachrichten über grosse Entfernungen übertragen werden.



Keine grosse Überraschung, aber sowohl der Telex sowie auch der Morsecode sind heute von modernen Technologien überholt worden. Die Frage ist aber, warum war man mit diesen Lösungen nicht zufrieden?

Aufgabe 2

Telex und Morsecode ermöglichten die Übertragung von Nachrichten über grosse Distanzen. Doch beide Verfahren haben ihre Schwachstellen. Versuchen wir sie herauszufinden.

Beim Telex wird jedes Zeichen mit 5 Bits codiert und beim Morsecode sind die Zeichen unterschiedlich lang.

- Wie viele verschiedene Zeichen könne mit 5 Bits grundsätzlich dargestellt werden?
- Reicht diese Anzahl aus, um alle Zeichen darzustellen, die wir heute beim Schreiben benötigen?
- Sie möchten die Nachricht «GE» übertragen. Schreiben Sie die entsprechende Folge aus Punkten und Strichen auf.
- Könnte ein Empfänger anhand dieser Folge eindeutig die einzelnen Zeichen erkennen? Was müsste noch zusätzlich gesendet werden, damit die Eindeutigkeit gewährleistet ist?
- Welche Eigenschaften müsste eine neue Zeichencodierung besitzen, damit sie die Schwachstellen von Telex und Morsecode möglichst gut löst?

1.1. ASCII Codierung

1963 wurde schliesslich der ASCII-Code eingeführt. Dieser verzichtete auf Start- und Stoppbits und wurde ursprünglich für alle Zeichen einheitlich auf 7 Bit festgelegt.

Diese Zeichen beinhalten:

- 33 nicht druckbare Steuerzeichen
- 95 echte Zeichen (siehe rechts)

	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	q	r	s	t	u	v	w	x	y	z	{		}	~	

Diese Codes für jedes einzelne Zeichen müssen Sie nicht auswendig lernen. Für das gibt es ASCII-Tabellen, welche Sie auch an der Prüfung benutzen dürfen. Schauen wir uns eine solche Tabelle konkret an ein paar Beispielen an:

ASCII-Tabelle lesen

In der Tabelle sehen Sie, dass dem Buchstaben A der Wert 65 zugeordnet ist. Diese Zuordnung wurde festgelegt und jedes Zeichen erhält dadurch einen eindeutigen Code. Das kleine a hat zum Beispiel den Wert 97. Das sind Dezimalzahlen. Der Computer speichert die Informationen jedoch nicht als Dezimalzahlen, sondern binär.

Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
32	20	040	 	Space	64	40	100	@	@	96	60	140	`	`
33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
36	24	044	$	\$	68	44	104	D	D	100	64	144	d	d
37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
38	26	046	&	&	70	46	106	F	F	102	66	146	f	f

In der Tabelle finden Sie deshalb auch den hexadezimalen Wert. Die Hexadezimalschreibweise ist kompakter und benötigt weniger Platz als die vollständige binäre Darstellung. Beim Buchstaben A ist der hexadezimale Wert beispielsweise 41₁₆. In Binärschreibweise entspricht dies: 100'0001₂ (7 Bit!).

5	5	005	ENQ	{enquiry}
6	6	006	ACK	{acknowledge}
7	7	007	BEL	{bell}
8	8	010	BS	{backspace}
9	9	011	TAB	{horizontal tab}
10	A	012	LF	{NL line feed, new line}

Die sichtbaren Zeichen beginnen in der Tabelle erst ab dem Wert 32. Davor befinden sich sogenannte Steuerzeichen. Diese lösen bestimmte Aktionen aus. Einige davon sind Ihnen wahrscheinlich bereits bekannt: Mit backspace löschen Sie beispielsweise ein Zeichen, und mit new line erzeugen Sie einen Zeilenumbruch.

Für die folgenden Aufgaben brauchen Sie eine ASCII-Tabelle, die Sie direkt unterhalb einsehen können.

Aufgabe 3

Schreiben Sie den folgenden Satz in ASCII, indem Sie die Codes in **dezimaler** Schreibweise hinschreiben:

Macht Informatik Spaß?

Aufgabe 4

Schreiben Sie den folgenden Satz in ASCII, indem Sie die Codes in **binärer** Schreibweise hinschreiben:

Heute ist es schön 😊

ASCII-Tabelle

Dec	Hex	Character
0	00	NUL
1	01	SOH
2	02	STX
3	03	ETX
4	04	EOT
5	05	ENQ
6	06	ACK
7	07	BEL
8	08	BS
9	09	HT
10	0A	LF
11	0B	VT
12	0C	FF
13	0D	CR
14	0E	SO
15	0F	SI
16	10	DLE
17	11	DC1
18	12	DC2
19	13	DC3
20	14	DC4
21	15	NAK
22	16	SYN
23	17	ETB
24	18	CAN
25	19	EM
26	1A	SUB
27	1B	ESC
28	1C	FS
29	1D	GS
30	1E	RS
31	1F	US
32	20	(Space)
33	21	!
34	22	"
35	23	#
36	24	\$
37	25	%
38	26	&
39	27	'
40	28	(
41	29	)
42	2A	*
43	2B	+
44	2C	,
45	2D	-
46	2E	.

Dec	Hex	Character
47	2F	/
48	30	0
49	31	1
50	32	2
51	33	3
52	34	4
53	35	5
54	36	6
55	37	7
56	38	8
57	39	9
58	3A	:
59	3B	;
60	3C	<
61	3D	=
62	3E	>
63	3F	?
64	40	@
65	41	A
66	42	B
67	43	C
68	44	D
69	45	E
70	46	F
71	47	G
72	48	H
73	49	I
74	4A	J
75	4B	K
76	4C	L
77	4D	M
78	4E	N
79	4F	O
80	50	P
81	51	Q
82	52	R
83	53	S
84	54	T
85	55	U
86	56	V
87	57	W
88	58	X
89	59	Y
90	5A	Z
91	5B	[
92	5C	\
93	5D	]

Dec	Hex	Character
94	5E	^
95	5F	_
96	60	`
97	61	a
98	62	b
99	63	c
100	64	d
101	65	e
102	66	f
103	67	g
104	68	h
105	69	i
106	6A	j
107	6B	k
108	6C	l
109	6D	m
110	6E	n
111	6F	o
112	70	p
113	71	q
114	72	r
115	73	s
116	74	t
117	75	u
118	76	v
119	77	w
120	78	x
121	79	y
122	7A	z
123	7B	{
124	7C	
125	7D	}
126	7E	~
127	7F	DEL

ASCII scheint also ebenfalls nicht ganz vollständig unsere modernen Anforderungen zu erfüllen. Zwei der Zeichen aus den Sätzen von den Aufgaben sind nicht in der Tabelle enthalten... Haben Sie eine Ahnung, warum Zeichen mit Akzent (é, â, ö, ñ) nicht enthalten sind? Die Antwort liegt im A von ASCII.

1.2. Weitere Codierungen

Eine naheliegende Idee war, ASCII zu erweitern: Statt 7 Bits verwendet man 8 Bits. Genau das machte IBM beispielsweise mit Code Page 437. Die ersten 128 Zeichen blieben unverändert und entsprachen ASCII. Die zusätzlichen 128 Zeichen wurden mit weiteren Buchstaben, Symbolen und grafischen Zeichen gefüllt.

Aufgabe 5

Wie viele verschiedene Zeichen können mit 8 Bits dargestellt werden? Wie viele davon kommen im Vergleich zu 7 Bits hinzu?

Wenn wir uns die Code Page 437 in Abbildung 4 genauer betrachten, können wir schnell überprüfen, dass die Zeichen für die ASCII Werte dieselben sind wie vorher:

Nehmen wir mal den Grossbuchstaben A. In der Tabelle sehen wir, dass er in Zeile 4, Spalte 1 ist. Das kann man so lesen, dass der Hexadezimalwert von A 41_{16} ist. Das ist derselbe Wert, den A in der Tabelle in Abbildung 3 hat.

Code Page 437

	-0	-1	-2	-3	-4	-5	-6	-7	-8	-9	-A	-B	-C	-D	-E	-F	
0-	*	@	Q	W	4	5	6	7	8	9	:	;	<	=	>	?	0-
1-	!	"	#	\$	%	&	'	(	)	*	+	,	-	.	/		1-
2-																	2-
3-	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?	3-
4-	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O		4-
5-	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^		5-
6-	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n		6-
7-	p	q	r	s	t	u	v	w	x	y	z	{		}	~		7-
8-	G	ü	é	â	ä	à	â	ç	ê	ë	è	é	î	ï	ñ		8-
9-	É	æ	ff	ô	ù	ò	û	ü	ç	û	ç	û	ç	û	ç		9-
A-	á	í	ó	ú	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ	ñ		A-
B-																	B-
C-																	C-
D-																	D-
E-																	E-
F-																	F-

Used first in IBM Monochrome Display Adapter in 1981.

Abbildung 4: Code Page 437, wie sie IBM für ihren Computer 1981 darstellte. Die Reihen und Spalten sind Hexadezimalzahlen.

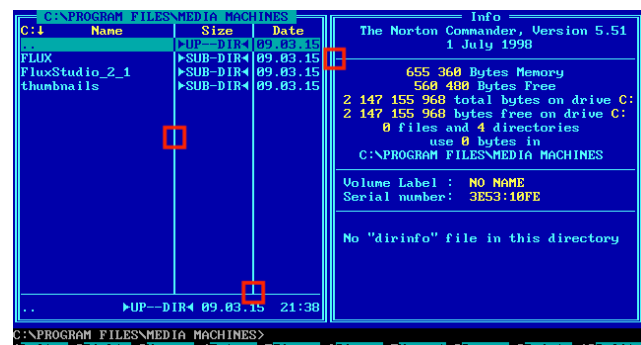


Abbildung 5: Der Norton Commander war eine frühere Version ihres heutigen Explorers/Finders auf ihrem Windows/Mac.

Da ASCII nicht ausreichte für einen grossen Teil der Weltbevölkerung, hat man das Konzept weiterentwickelt. Zuvor hat man die Kosten von einem weiteren Bit gescheut, sich aber dann schlussendlich doch dafür entschieden, auf 8 Bit zu erweitern. Diese Systeme hatten dann andere Namen (ANSII, Codepage 437, Codepage 850, Unicode, ...).

Codepage 437:

IBM hatte 1981 bei der Einführung ihres Computers vorgehabt, ASCII zu nehmen und dann gemerkt, dass wenn man in der Schweiz oder anderen europäischen Ländern den Computer verkaufen möchte, man wohl oder übel Umlaute braucht. Und darum haben die zwar ASCII genommen aber dann 1 Bit dazugenommen und somit von 7 Bit auf 8 Bit erweitert. Was bedeutet das? Wie viele zusätzliche Zeichen kriegen wir dann? Zusätzlich 128 Zeichen! (Aufgabe)

(Zeige hier die Tabelle von Codepage 437 mit dem Vergleich, was 1 Bit zusätzlich ausmacht.)

Also mit einem Bit hat man all die unteren Zeichen dazugewonnen inklusive all die mit den Umlauten. Sowie weitere Zeichen, die IBM damals als sinnvoll angeschaut hat. Diese komischen Linien wurden z.B. gebraucht, um Linien zu zeichnen. Also Grafiken zu erstellen, was sonst nicht möglich war.

(Beispiel vom Norton Commander, wo man die Linien sieht)

Das Problem bleibt halt immer noch, dass andere Sprachen schwieriger waren.. Deutsch war okay, aber dänisch oder spanisch wird schon schwieriger. Drum ein paar Jahre später ist Microsoft ein paar Schritte weitergegangen und hat eine Codepage 850 erstellt, welche spezifisch auf westeuropäische Sprachen spezialisiert war. Also all die speziellen Linien zeichnen, wurden dann effektiv mit Zeichen ersetzt, welche in westeuropäischen Sprachen benutzt werden. Speziell: Griechisch und Kyrillisch haben gefehlt... Darum hat man ganz viele Codepages definiert.. z.B. eine andere Codepage war extra für mitteleuropäisch und sollte albanisch, kroatisch, polnisch etc. beinhalten.

Die Folge: Eine lange Liste von verschiedenen Codepages... (insert Liste von Codepages)

Aber das Problem besteht... Wenn ich meinen Computer auf westeuropäisch eingestellt habe und einen Brief auf Word schreibe. Diesen verschicke in ein Land, wo z.B. Kyrillisch verwendet wird. Dann konnte nicht sichergestellt werden, dass die Person am anderen Ende diesen Brief in sinnvollen Zeichen darstellen konnte...

insert Beispielbild, wo Zeichen falsch angezeigt werden, z.B. grad bei Norton Commander wenn die Linien als Buchstaben mit Umlauten angezeigt werden.

Schlussendlich blieb das ganze ein riesiges Chaos. Die ersten 128 Zeichen blieben ASCII aber der Rest hat sich überall unterschieden. Wie konnte man das ganze vereinheitlichen?

1.3. UTF-8 Codierung

2. Bildercodierung

Lösungen

Lösung 1

$\frac{150}{7.5} \text{ ms} = 20 \text{ ms}$ für die ersten 6 Impulse und 30ms für den Stoppimpuls.

Eine Sekunde hat 1000ms, dh. pro Sekunde können $\frac{1000}{150} \approx 6.67$ Zeichen geschrieben werden. Pro Minute bedeutet das 400 Zeichen. Das ist nicht die schnellste Technologie aber sicher auch nicht so langsam, wie man sich das vielleicht vorstellt!

Lösung 2

- a) Bei 5 Bit gibt es $2^5 = 32$ Möglichkeiten. Das heisst 32 verschiedene Zeichen sind möglich.
- b) Nein. 32 Zeichen reichen nicht aus, um beispielsweise alle Gross- und Kleinbuchstaben, Zahlen, Satzzeichen und weitere Sonderzeichen (Emojis?) darzustellen.
- c) G (— ·) und E (·) zusammen ergibt — · ·
- d) Nein. Die Folge — · · könnte als GE verstanden werden, aber auch als Buchstabe Z. Der Empfänger weiss ohne zusätzliche Information also nicht, wo das G endet und das E beginnt. Es müsste also eine Pause oder ein zusätzliches Trennsignal zwischen den Zeichen gesendet werden.
- e) Eine geeignete (moderne) Zeichencodierung müsste genügend verschiedene Zeichen darstellen können und eindeutig sein, damit der Empfänger weiss, wo ein Zeichen beginnt und endet.

Lösung 3

77 97 99 104 116 32 73 110 102 111 114 109 97 116
105 107 32 83 112 97 115 115 63

Lösung 4

1001000 1100101 1110101 1110100 1100101 0100000
1101001 1110011 1110100 0100000 1100101 1110011
0100000 1110011 1100011 1101000 1101111 1100101
1101110

Für das ö haben wir kein geeignetes ASCII-Zeichen gesehen, darum haben wir es mit oe ersetzt. Für das Emoji haben wir aber bisher keine Lösung...

Lösung 5

$2^7 = 128$ und $2^8 = 256$. Es kommen also 128 neue Zeichen dazu (doppelt so viele wie vorher).